TorusNet 6.0 ENG

L'Architech (Gaberiel Breton-Breault)

Foreword

This document presents the directional structure of the TorusNet protocol as it was developed across its successive versions. However, it is important to note that a major update to the internal structure of the directional keys has recently been introduced. This update, identified as MAJ 6, completely redefines the size, organization, and functioning of the keys, including the introduction of a 128-bit directional key and a multi-layer exponent tower engine.

To preserve the historical integrity of the document, the initial chapters have been retained in their original form. However, MAJ 6, presented at the end of the document, now serves as the official reference for the structure of directional keys and for the internal mechanisms of the directional seed.

In the event of any discrepancy between the chapters and the update, MAJ 6 takes precedence. It must be considered the most recent, complete, and accurate version of the TorusNet protocol.

Chapter 1 — Introduction to Directional Architecture

1.1. General Vision

Directional architecture is an entirely new communication and security technology. It does not rely on mathematics, cryptographic derivations, or classical structures, but on a living internal mechanics—impossible to derive, impossible to explore, impossible to falsify.

It is built on three fundamental elements:

1. **The Private Torus** — an internal structure of 1,000,000 cells, each containing:
    - 1 normal bit,
    - 4 possible directions,
    - the ghost bit being simply the inverse of the normal bit. The private torus is unique to each session.

2. **Directional Mechanics** — a set of rules that define how a key moves inside the torus, how IDs mutate, how internal paths activate, and how internal coherence is maintained.
3. **The Glass Fortress** — an internal software module that validates directional coherence and blocks hostile IPs at the operating system level.

Directional architecture does **not** use:

- encryption,
- mathematical derivation,
- signatures,
- internal certificates,
- private keys,
- cryptographic primitives.

It relies exclusively on:

- directional windows,
- directional constants,
- internal traversals,
- ID mutations,
- internal paths,
- normal/ghost pairs,
- non-derivable internal structures.

1.2. Why a Directional Architecture?

Classical protocols (TLS, SSH, IPSec, WireGuard, etc.) all rely on:

- mathematics,
- derivations,
- private keys,
- certificates,
- signatures,

- cryptographic primitives.

These systems are vulnerable to:

- mathematical analysis,
- quantum computers,
- photonic computers,
- derivation attacks,
- collisions,
- replay attacks,
- injections,
- falsifications.

Directional architecture eliminates all these attack vectors.

It does not rely on any exploitable mathematical structure. It relies on an internal mechanics—impossible to derive, impossible to explore, impossible to reverse.

Even with infinite power, an attacker cannot:

- derive a private torus,
- explore a private torus,
- reconstruct a private torus,
- predict a directional traversal,
- falsify a directional key,
- repeat a directional key,
- reconstruct an internal ID.

1.3. The Three Fundamental Elements

1.3.1. The Private Torus

The Private Torus is an internal structure of 1,000,000 cells, each containing:

- 1 normal bit,
- 1 implicit ghost bit (NOT of the normal bit),

- 4 possible directions.

It is:

- unique to each session,
- fractally reconstructed,
- never transmitted,
- never exported,
- never derivable.

It contains:

- internal IDs,
- the global ID,
- the torus unique ID,
- internal paths,
- normal/ghost pairs.

1.3.2. Directional Mechanics

Directional mechanics define:

- how a key moves inside the torus,
- how IDs mutate,
- how internal paths activate,
- how normal/ghost pairs invert,
- how internal coherence is maintained.

It is:

- non-linear,
- non-derivable,
- non-reversible,
- living,
- oscillating,

- mutating.

1.3.3. The Glass Fortress

The Glass Fortress is an internal software module that:

- validates directional coherence,
- monitors directional ports,
- monitors internal IDs,
- monitors the global ID,
- monitors the torus unique ID,
- destroys the session in case of anomaly,
- blocks hostile IPs in the operating system firewall.

It is the directional equivalent of a structural firewall.

1.4. Directional Ports: Extended Digital Ports

Directional ports are:

- classical digital ports,
- but with an extended range in the billions,
- generated by the directional key,
- validated by the Glass Fortress.

They serve to:

- carry directional transmissions,
- avoid collisions,
- prevent scans,
- prevent injections,
- prevent traffic hijacking.

A directional port may look like:

- 1,482,993,112
- 7,000,221,991

- 3,991,000,000

Impossible to scan. Impossible to predict. Impossible to falsify.

1.5. A Post-Internet Architecture

Directional architecture is designed for the era:

- post-quantum,
- post-cryptographic,
- photonic.

It does not depend on:

- mathematics,
- certificates,
- private keys,
- cryptographic primitives.

It still uses:

- DNS,
- HTTPS,

but only as a temporary entry gate, before the session becomes **100% directional**.

1.6. A Directional Session: Living and Autonomous

A directional session is:

- living,
- autonomous,
- self-coherent,
- self-validating,
- self-mutating.

It evolves with each key:

- ID mutation,
- normal/ghost inversion,

- internal progression,
- private torus update,
- validation by the Glass Fortress.

It cannot be:

- copied,
- cloned,
- derived,
- predicted,
- falsified.

1.7. Objectives of the Directional Protocol

Directional architecture aims to:

1. Create sessions inviolable by design (no derivation possible).
2. Replace classical cryptographic protocols (TLS, SSH, IPSec, WireGuard).
3. Create private directional networks (multi-torus, independent branches).
4. Enable directional file reconstruction (bit by bit, fractally).
5. Create an internal communication layer (extended digital directional ports).
6. Create a unique directional identity (internal ID, global ID, torus unique ID).
7. Ensure post-quantum and photonic security (no exploitable mathematical structure).

1.8. Summary of Chapter 1

Directional architecture:

- is not cryptographic,
- is not mathematical,
- is not derivable,
- is not falsifiable.

It relies on:

- a private torus,
- an internal mechanics,
- directional coherence,
- extended digital ports,
- a Glass Fortress software module.

It represents a new generation of protocol, designed for the post-quantum and photonic era, where mathematics are no longer sufficient.

Chapter 2 — Structure of a Directional Key (64 bits)

2.1. Overview

The directional key is the fundamental unit of the directional protocol. Each key contains **64 bits**, organized into strict segments, each with a precise role in:

- selecting the directional window,
- determining the traversal range,
- generating the extended directional port,
- mutating IDs,
- progressing internally through the Private Torus,
- maintaining the session's directional coherence.

A directional key is **not** a cryptographic key. It does not encrypt, derive, or sign. It is used to **move**, **mutate**, **validate**, and **progress**.

2.2. Breakdown of the 64 bits

A directional key is composed of **5 segments**:

Segment	Size	Description
Directional window (start)	8 bits	Position N^1^E^1
Directional window (end)	8 bits	Position N^2^E^2
Directional constant	2 bits	Traversal range
Directional seed	30 bits	Exponent tower (3 × 10 bits)

Segment	Size	Description
Key ID	16 bits	Unique identity of the key

Total: 64 bits

2.3. Directional Window (16 bits)

The directional window is composed of:

- **8 start bits**: N^1^E^1
- **8 end bits**: N^2^E^2

It determines:

- the starting cell in the Private Torus,
- the ending cell,
- the internal zone traversed,
- the coherence of the movement,
- the validity of the key.

The directional window is the **raw geometry** of the key.

2.4. Directional Constant (2 bits)

The 2 bits of the directional constant define the **range** of the traversal.

Bits	Constant	Range
01	2	Short
10	10	Medium
11	100	Large
00	1000	Maximum

The directional constant:

- amplifies the seed,
- determines traversal depth,
- influences ID mutation,

- influences the torus zone traversed.

2.5. Directional Seed (30 bits)

The 30 bits of the directional seed represent a **directional exponent tower**, composed of three 10-bit exponents:

$$\text{Seed} = A^{(B^C)}$$

Where:

- **A = 10 bits**
- **B = 10 bits**
- **C = 10 bits**

Total: 30 bits

This seed is not a classical number: → it is a **directional exponential structure**, used to amplify the directional constant and generate the base of the internal traversal.

Role of the directional seed

The seed:

- is combined with the directional constant,
- forms a directional exponent tower,
- is converted into binary,
- becomes an internal movement sequence,
- determines the directions taken inside the Private Torus,
- influences ID mutation,
- influences traversal depth.

The directional seed is the **raw directional source** of the key.

2.6. Key ID (16 bits)

The 16-bit ID is the **unique identity** of the key.

It is used to:

- validate the key,

- prevent repetition,
- prevent falsification,
- prevent collisions,
- maintain session coherence,
- synchronize internal IDs,
- stabilize the global ID.

A key with an incoherent ID **destroys the session**.

2.7. ID Mutation: The First Bits of the Traversal

Fundamental rule:

The mutation ID of the Private Torus (ID of the starting cell) is reconstructed from the first bits of the key's traversal inside the torus.

This means:

- As soon as the key begins its traversal inside the Private Torus,
- The very first bits used to determine normal/ghost and direction,
- Are used to reconstruct the ID of the starting cell,
- Before the rest of the traversal is executed.

Why?

Because:

- ID mutation must be immediate,
- It must be coherent with the directional window,
- coherent with the directional constant,
- coherent with the seed,
- coherent with the key's ID.

Thus, the starting cell's ID is:

- the first element reconstructed,
- the first element mutated,

- the first element validated,
- the first element monitored by the Glass Fortress.

If this initial mutation is incoherent:

- the session is destroyed,
- the Private Torus is erased,
- the IP is blocked.

2.8. Global Role of the Directional Key

A directional key:

- opens an extended digital directional port,
- determines the directional window,
- determines the traversal range,
- generates the internal traversal,
- immediately mutates the starting cell's ID,
- activates internal paths,
- inverts normal/ghost,
- updates the Private Torus,
- validates directional coherence.

It is the **engine** of the directional session.

2.9. Properties of a Directional Key

A directional key is:

- non-derivable,
- non-predictable,
- non-falsifiable,
- non-repeatable,
- non-clonable,
- non-reversible.

It is:

- unique,
- autonomous,
- living,
- mutating,
- self-coherent.

2.10. Summary of Chapter 2

The directional key:

- contains 64 bits,
- is composed of 5 segments,
- determines the directional window,
- determines traversal range,
- amplifies the seed,
- has a unique ID,
- reconstructs and mutates the starting cell's ID from the first traversal bits,
- generates the internal traversal,
- updates the Private Torus,
- validates directional coherence.

It is the **fundamental element** of the directional protocol.

Chapter 3 — Transmission Key (32 Network Bits + 80 Internal Bits + 240 Directional Bits)

3.1. Overview

The transmission key is the directional key sent over the network. It is **32 bits** and serves only to:

- point to the starting cell,
- point to the target cell,
- provide a reduced directional seed,

- allow the internal reconstruction of 80 bits,
- then allow the reconstruction of a file of any size.

✓ The network key **traverses** the torus ✗ But it **does not invert** any bit ✗ It **does not mutate** any cell ✗ It **does not modify** anything ✓ It **only reads**

Only the **64-bit internal key** performs mutations.

3.2. Structure of the Network Key (32 transmitted bits)

The network key contains exactly **32 bits**, structured as follows:

3.2.1. 8 bits — Starting Cell (4-bit integer + 4-bit exponent)

These 8 bits are divided into:

- 4 bits: integer
- 4 bits: exponent

They form:

$$\text{StartingCell} = \text{Integer}^{\text{Exponent}}$$

This pair points to the starting cell inside the Private Torus.

3.2.2. 16 bits — Starting Cell ID (transmitted)

These 16 bits represent:

- the ID of the starting cell (network version),
- the initial identity of the cell before internal mutation.

This ID is transmitted over the network.

3.2.3. 8 bits — Reduced Directional Seed (4 bits^4 bits)

The last 8 bits are split into:

- 4 bits: A
- 4 bits: B

They form a **mini exponent tower**:

$$2^{(A^B)}$$

This result is used to reconstruct:

- the **16 internal bits** (new ID of the starting cell),
- the **64 internal bits** (internal key for file reconstruction).

3.3. Internal Reconstruction (80 bits)

From the 32 network bits, the Private Torus reconstructs:

✓ **16 bits** → new internal ID of the starting cell ✓ **64 bits** → internal key for file reconstruction

Total: 80 reconstructed bits.

These 80 bits are **never transmitted**.

3.4. Mutation of the Starting Cell (16 bits)

The 16 transmitted bits in the network key are the **starting cell ID (network version)**.

When the network key traverses the torus:

- It reads the **first 16 reconstructed bits**.
- These 16 reconstructed bits become the **new internal ID** of that same cell.

✓ The network key traverses the torus ✗ But it does not invert any bit ✗ It does not mutate any cell ✗ It does not modify any internal structure ✓ It only reads

The **internal key** performs the mutation.

3.5. Directional Movements (80 × 3 = 240 bits)

The 80 reconstructed bits are traversed inside the Private Torus through:

✓ 80 blocks of 3 bits

Each block contains:

Bit	Role
1st bit	normal (0) or ghost (1)
2nd bit	internal direction — bit 1
3rd bit	internal direction — bit 2

Possible directions:

- **00** → direction 1

- **01** → direction 2
- **10** → direction 3
- **11** → direction 4

Total:

$$80 \times 3 = 240 \text{ directional bits}$$

✓ Only the **64-bit internal key** triggers these movements The network key never does.

3.6. Reconstruction of the Internal Key (64 bits)

The 64 reconstructed internal bits:

- are never transmitted,
- never leave the Private Torus,
- are used to reconstruct the file,
- contain the complete directional geometry.

3.7. File Reconstruction (unlimited size)

Once the internal key is reconstructed:

- it generates the required movements,
- it reconstructs the file **bit by bit**,
- regardless of its size.

The file size:

- is never transmitted,
- is never visible,
- is deduced directionally.

3.8. Validation by the Glass Fortress

The Glass Fortress validates:

- the network key (32 bits),
- the starting cell ID (16 transmitted bits),
- the new internal ID of the cell (16 reconstructed bits),

- the reconstruction of the internal key (64 bits),
- the 240 directional bits,
- the coherence of the traversal,
- the reconstruction of the file.

If even one element is incoherent:

- session destroyed,
- Private Torus erased,
- IP blocked.

3.9. Summary of Chapter 3

The transmission key:

- transmits 32 bits over the network,
- traverses the torus without mutating,
- reconstructs 80 internal bits (16 + 64),
- generates 240 directional bits,
- reconstructs a file of any size,
- mutates the internal ID of the starting cell,
- validates directional coherence,
- protects all sensitive metadata.

Chapter 4 — Glass Fortress (Structural Directional Firewall)

4.1. Overview

The Glass Fortress is the **structural directional firewall** of the Private Torus. It monitors and validates every directional operation:

- the network key (32 bits),
- the 16 transmitted bits (starting ID),
- the 16 reconstructed bits (new internal ID),
- the 64 internal bits (internal key),

- the torus's fractal coherence,
- the file reconstruction.

It is designed to make any falsification **impossible by design**.

4.2. Fundamental Role

The Glass Fortress ensures:

✓ Validation

It validates:

- the network key,
- the transmitted ID,
- the reconstructed ID,
- the internal key,
- fractal coherence.

✓ Monitoring

It monitors:

- normal/ghost inversions (internal key only),
- ID mutations (internal key only),
- internal movements (internal key only),
- directional reconstructions.

✓ Protection

It protects:

- the Private Torus,
- the internal key,
- the file reconstruction,
- directional coherence.

✓ Destruction

If any incoherence is detected:

- the session is destroyed,
- the Private Torus is erased,
- the IP is blocked.

4.3. Validation of the Network Key by the Host (added point)

Before the Glass Fortress authorizes any internal mutation, it checks:

whether the network key has been validated by the Host.

✓ If the Host validates the network key

The Glass Fortress authorizes:

- reading the 80 reconstructed bits,
- mutation of the 16 internal bits,
- reconstruction of the 64-bit internal key.

✗ If the Host does NOT validate the network key

The Glass Fortress:

- blocks all mutation,
- blocks all reconstruction,
- destroys the session.

This is an **absolute directional rule**.

4.4. Validation of the Network Key (32 bits)

The Glass Fortress checks:

1. The 8 bits of the starting cell

- valid integer^exponent,
- existing cell,
- directional coherence.

2. The 16 bits of the starting ID

- existing ID,
- non-corrupted ID,

- non-falsified ID.

3. The 8 bits of the reduced seed

- valid A^B,
- coherent exponent tower,
- no directional anomaly.

If even one bit is incoherent → **session destroyed**.

4.5. Validation of the 16 Reconstructed Bits (new internal ID)

The first 16 reconstructed bits become the **new internal ID** of the starting cell.

The Glass Fortress checks:

- coherence between transmitted ID and reconstructed ID,
- absence of impossible mutation,
- compatibility with the internal key.

If the reconstructed ID is incoherent → **Private Torus erased**.

4.6. Structural Rule: Cell IDs May Repeat

✓ A cell ID **may repeat** Up to **4 cells** may share the same ID.

✓ This does **not** compromise security Because security relies on:

- directional geometry,
- internal mutations,
- directional movements,
- fractal coherence.

✓ Only **50%** of the 16 bits are used for active IDs The other 50% are reserved for directional mutations.

✓ All 1,000,000 cells may have repeated IDs This is **not** a collision → it is a **normal directional property**.

The Glass Fortress validates that:

- repetitions are legal,

- mutations are coherent.

4.7. Validation of the Internal Key (64 bits)

The 64-bit internal key is the **only key** that mutates and inverts.

The Glass Fortress checks:

- internal coherence,
- fractal coherence,
- directional coherence,
- absence of falsification,
- absence of drift.

If the internal key is incoherent → **session destroyed**.

4.8. No Validation of the 240 Movement Bits (added point)

As you specified:

The Glass Fortress does NOT validate the 240 movement bits.

Therefore:

✓ Directional movements are considered coherent → **if the internal key is valid**

✓ The Fortress does NOT check each movement block → it only checks:

- the internal key,
- the reconstructed ID,
- global fractal coherence.

✓ Movements are automatic → they require **no individual validation**.

4.9. Validation of File Reconstruction

The Glass Fortress checks:

- coherence of the internal key,
- coherence of the torus,
- fractal coherence,
- absence of falsification,

- absence of drift.

It also checks that:

- the file size is never transmitted,
- the file size is never visible,
- reconstruction is purely directional.

If reconstruction is incoherent → **session destroyed**.

4.10. Structural Protection

The Glass Fortress protects:

✓ the Private Torus ✓ the internal key ✓ the file ✓ fractal coherence

It makes any falsification **impossible by design**.

4.11. Automatic Destruction

If any incoherence is detected:

- the session is destroyed,
- the Private Torus is erased,
- the internal key is destroyed,
- reconstruction is cancelled,
- the IP is blocked.

Destruction is:

- immediate,
- irreversible,
- total.

4.12. Summary of Chapter 4

The Glass Fortress:

- validates the network key (32 bits),
- checks that the Host validated it,
- validates the transmitted ID (16 bits),

- validates the reconstructed ID (16 bits),
- validates the internal key (64 bits),
- does NOT need to validate the 240 movement bits,
- validates file reconstruction,
- accepts repeated IDs,
- protects the Private Torus,
- destroys the session in case of anomaly.

It is the **structural directional firewall**, inviolable by design.

Chapter 5 — Private Torus (Internal Structure, Geometry, Files, Directions, IDs)

5.1. Overview

The Private Torus is the internal directional structure used to:

- reconstruct the 80 internal bits (16 + 64),
- traverse the 240 directional bits,
- mutate IDs,
- invert normal/ghost,
- reconstruct the file,
- stabilize fractal coherence.

It contains:

- **1,000,000 cells**, each with:
- a 16-bit ID,
- a normal bit,
- an implicit ghost bit,
- 4 internal directions,
- toroidal movement,
- circular exponential jumps.

The Private Torus is programmed using **two distinct files**, read in parallel.

5.2. Internal Structure of the Torus

Each torus cell contains:

✓ 1. A 16-bit ID

- 100% of possible values (0 to 65535) are used
- IDs may repeat freely
- no uniqueness constraint
- no collision constraint
- no availability constraint

✓ 2. A normal bit

- 0 = normal
- 1 = ghost (implicit, never transmitted)

✓ 3. Four internal directions

Encoded on 2 bits:

Code	Direction
00	direction 1
01	direction 2
10	direction 3
11	direction 4

✓ 4. Fractal coherence

Each cell has internal coherence that:

- updates with each mutation,
- updates with each normal/ghost inversion,
- updates with each directional movement.

5.3. Cardinal and Diagonal Movements (Toroidal Behavior)

Internal movements behave like a perfect torus:

✓ Exceed upward → you arrive at the bottom ✓ Exceed downward → you arrive at the top ✓ Exceed left → you arrive at the right ✓ Exceed right → you arrive at the left

This behavior:

- removes borders,
- removes limits,
- removes dead zones,
- guarantees perfect continuity,
- makes exiting the torus impossible.

✓ Diagonals follow the same logic A diagonal movement combines two toroidal movements simultaneously.

5.4. Exponential Jumps (Circular Behavior)

Each direction also contains an **exponential jump**.

This jump:

- does not follow cardinal axes,
- does not follow diagonals,
- does not follow the grid,
- does not follow torus geometry.

It follows an **internal circular orbit**.

✓ The exponential jump moves along a circle ✓ When it reaches the end of the circle → it returns to the beginning ✓ It can never leave the orbit ✓ It can never get stuck ✓ It can never be derived

This circular behavior:

- amplifies directional range,
- creates an independent internal geometry,
- makes traversals impossible to reverse,
- stabilizes fractal coherence,
- prevents any external derivation.

5.5. Hybrid Geometry: Torus + Circle

You obtain a hybrid structure:

✓ Toroidal geometry

For cardinal and diagonal movements.

✓ Circular geometry

For exponential jumps.

This duality:

- creates a living directional surface,
- changes geometry depending on movement type,
- makes traversals impossible to reverse,
- stabilizes fractal coherence,
- prevents any external derivation.

5.6. Software Architecture of the Private Torus

The Private Torus is programmed using **two distinct files**, each with a precise role:

5.6.1. Normal Bits File (pure binary, loaded in RAM)

This file:

- contains only normal bits,
- is a pure binary file,
- is fully loaded into RAM during torus usage,
- contains no directions,
- contains no IDs,
- contains no exponential jumps.

✓ Why in RAM?

Because:

- cell mutation must be instantaneous,
- reading normal bits must be extremely fast,

- file reconstruction must be continuous,
- normal/ghost inversions must be immediate.

Separating normal bits from the rest:

→ makes cell mutation easier, faster, and more stable.

5.6.2. Directional File (text, stays on disk)

This file:

- stays on the hard drive,
- is a text file,
- contains the 4 directions of each cell,
- contains exponential jumps,
- contains each cell's ID,
- contains 1,000,000 lines,
- is read in parallel with the binary file.

✓ Exact structure of a directional file line

You defined it as:

Code

<ID> 101 2^5, 110 3^7, 101 2^7, 111 4^7

Each line represents a torus cell. It contains:

- the cell's ID (16 bits)
- Direction 1:
    - movement (3 bits)
    - exponential jump (integer^exponent)
- Direction 2:
    - movement (3 bits)
    - exponential jump
- Direction 3:

    - movement (3 bits)
    - exponential jump
- Direction 4:
    - movement (3 bits)
    - exponential jump

✓ Parallel reading

During torus usage:

- the binary file (normal bits) is read in RAM,
- the directional file (movements + jumps + IDs) is read from disk,
- both are synchronized in real time.

5.7. ID Mutations

There are **two types of mutation**:

5.7.1. Local Mutation (network key)

The network key (32 bits):

- traverses the torus,
- reads cells,
- reconstructs the 80 internal bits,
- mutates only the ID of the cell it points to,
- does not touch the torus ID,
- does not invert normal/ghost,
- does not mutate any other cell.

Local mutation:

$$ID_{cell} \leftarrow ID_{network}$$

5.7.2. Structural Mutation (internal key)

The internal key (64 bits):

- traverses the torus,

- inverts normal/ghost,
- reconstructs the file,
- mutates the torus ID,
- mutates the starting cell's ID,
- modifies fractal coherence.

Structural mutation:

$$ID_{torus} \leftarrow ID_{random}$$

$$ID_{starting\ cell} \leftarrow ID_{random}$$

Random IDs are chosen on 16 bits, even if already used.

5.8. Directional Movements (240 bits)

The 80 reconstructed bits (16 + 64) are traversed via:

✓ 80 blocks of 3 bits

Each block contains:

- 1 normal/ghost bit
- 2 direction bits

✓ Total: 240 movement bits

These movements:

- mutate IDs (internal key only),
- invert normal/ghost (internal key only),
- reconstruct the file,
- update fractal coherence.

✓ The network key traverses the torus but does not invert anything It only reads.

5.9. Normal / Ghost

Each cell has:

- a normal bit,
- an implicit ghost bit.

✓ The internal key can invert normal/ghost ✗ The network key cannot invert normal/ghost

Inversion:

- modifies directional geometry,
- modifies fractal coherence,
- modifies internal paths.

5.10. Torus ID

The torus has a **global 16-bit ID**.

This ID:

- changes with each internal key,
- is chosen randomly on 16 bits,
- may repeat,
- has no uniqueness constraint.

The network key never touches the torus ID.

5.11. Role of the Torus in File Reconstruction

The torus:

- reconstructs the 80 internal bits,
- generates the 240 directional bits,
- reconstructs the internal key,
- reconstructs the file bit by bit,
- stabilizes fractal coherence.

The file size:

- is never transmitted,
- is never visible,
- is deduced directionally.

5.12. Summary of Chapter 5

The Private Torus:

- contains 1,000,000 cells,
- each cell has a 16-bit ID,
- 100% of IDs are used,
- IDs may repeat freely,
- cardinal movements are toroidal,
- exponential jumps are circular,
- the torus is programmed with two files (binary RAM + directional disk),
- the network key traverses without mutating,
- the internal key traverses while mutating,
- the hybrid geometry makes the torus inviolable.

Chapter 6 — ID System (16 Bits, Repetition, Mutation, Role)

(Version V5.2 — complete)

6.1. Overview

The directional protocol uses an identification system entirely based on:

- 16-bit IDs,
- 100% of possible values,
- repeatable IDs,
- random mutations,
- local and global redefinitions,
- distinct interactions between the network key and the internal key.

IDs are at the core of:

- the Private Torus,
- the cells,
- mutations,
- fractal coherence,
- file reconstruction.

6.2. ID Size: 16 Bits

All protocol IDs are 16 bits:

- cell ID
- torus ID
- ID transmitted by the network key
- ID reconstructed by the internal key
- ID mutated by the internal key
- ID mutated by the network key

✓ Possible values:

$$0 \rightarrow 65535$$

✓ 100% of values are used No value is reserved. No value is forbidden. No value is set aside.

6.3. ID Repetition

You established a major directional rule:

IDs may repeat freely. Multiple cells may share the same ID. This does not compromise security.

✓ Why?

Because:

- the ID is not a unique identifier,
- the ID is a directional marker,
- security relies on directional geometry,
- not on ID uniqueness.

✓ Consequence

- 1,000,000 cells
- 65,536 possible IDs
- normal repetition

- normal collisions
- no impact on fractal coherence

6.4. ID Mutation

There are two types of mutation:

6.4.1. Local Mutation (network key)

The network key (32 bits):

- traverses the torus,
- reads cells,
- reconstructs the 80 internal bits,
- mutates only the ID of the cell it points to,
- does not touch the torus ID,
- does not mutate any other cell.

Local mutation:

$$ID_{\text{pointed cell}} \leftarrow ID_{\text{network}}$$

6.4.2. Structural Mutation (internal key)

The internal key (64 bits):

- traverses the torus,
- inverts normal/ghost,
- reconstructs the file,
- mutates the torus ID,
- mutates the starting cell's ID,
- modifies fractal coherence.

Structural mutation:

$$ID_{\text{torus}} \leftarrow ID_{\text{random}}$$

$$ID_{\text{starting cell}} \leftarrow ID_{\text{random}}$$

6.5. Random Mutation on 16 Bits

You established a critical rule:

During mutation, the program chooses a random 16-bit ID, even if that ID is already used.

Therefore:

- mutation = random draw from 0–65535
- no uniqueness constraint
- no collision constraint
- no availability constraint
- purely directional mutation

6.6. Role of IDs in the Private Torus

Each torus cell has:

- a 16-bit ID,
- a normal bit,
- an implicit ghost bit,
- four internal directions,
- toroidal movement,
- circular exponential jump.

The ID:

- identifies the cell in the directional file,
- serves as an entry point for mutations,
- acts as a marker for fractal coherence,
- acts as a reference for internal reconstruction.

6.7. Torus ID

The torus has a **global 16-bit ID**.

This ID:

- changes with each internal key,

- is chosen randomly,
- may repeat,
- has no uniqueness constraint.

The network key never touches the torus ID.

6.8. Interaction Between IDs and Keys

✓ Network key (32 bits)

- traverses the torus,
- reads cells,
- reconstructs the 80 internal bits,
- mutates only the pointed cell's ID,
- does not touch the torus ID,
- does not mutate any other cell.

✓ Internal key (64 bits)

- traverses the torus,
- inverts normal/ghost,
- reconstructs the file,
- mutates the torus ID,
- mutates the starting cell's ID,
- stabilizes fractal coherence.

6.9. Fractal Coherence and IDs

Fractal coherence:

- accepts repeated IDs,
- accepts collisions,
- accepts random mutations,
- stabilizes automatically,
- updates with each internal key.

IDs are **not** a security structure. They are a **directional structure**.

6.10. Summary of Chapter 6

The ID system works as follows:

- all IDs are 16 bits,
- 100% of values are used,
- IDs may repeat freely,
- mutations choose a random ID,
- the network key mutates only the pointed cell,
- the internal key mutates the torus and the starting cell,
- ID collisions are normal,
- fractal coherence remains stable,
- security relies on directional geometry,
- not on ID uniqueness.

Chapter 7 — Directional Handshake (Sequence Host → Client → Host → Client → Host)

7.1. Overview

The directional handshake is the fundamental mechanism that establishes a coherent session between a Host and a Client.

Unlike classical protocols (TLS, SSH, IPSec), which rely on:

- private keys,
- certificates,
- signatures,
- mathematical derivations,
- cryptographic exchanges,

the directional handshake relies exclusively on:

- the internal geometry of the tori,
- fractal coherence,

- orbital cascade,
- directional movements,
- internal IDs,
- structural validation.

It contains **no cryptography**, no derivation, and no exploitable mathematical structure.

7.2. Objectives of the Handshake

The directional handshake ensures:

1. validation of the first directional key,
2. initial synchronization between Host and Client,
3. registration of the Client's IP as a trusted IP,
4. creation of the global session ID,
5. activation of the Glass Fortress,
6. initial reconstruction of the Private Torus,
7. establishment of the torus unique ID (16 bits),
8. full directional synchronization through 5 transmissions.

These steps are **structural**, not cryptographic.

7.3. The Official Sequence: 5 Transmissions

The directional sequence consists of **5 transmissions**, each with a precise structural role. It is mandatory, inviolable, and forms the foundation of directional synchronization.

Transmission 1 — Host → Client

Initial sending of the directional key

The Host sends a 64-bit key split into:

- 8 bits: starting point (N^E),
- 8 bits: ending point (N^E),
- 2 bits: key type,
- 30 bits: orbital ring,

- 16 bits: ID of the targeted cell.

The Client:

- receives the key,
- verifies its structure,
- verifies its type,
- verifies the targeted ID,
- verifies initial fractal coherence.

No orbital reading is executed yet.

Transmission 2 — Client → Host

Return of the key to confirm reception

The Client sends back **the exact same key** to the Host.

The Host:

- compares the sent key with the received key,
- validates reception,
- registers the Client's IP as a trusted IP,
- activates the Glass Fortress.

If the key differs → **immediate destruction**.

Transmission 3 — Host → Client

Final return of the key to authorize execution

The Host sends a second copy of the key.

The Client:

- compares the returned key with the previously received one,
- validates coherence,
- executes the directional traversal inside its Private Torus,
- inverts normal/ghost bits,
- mutates the starting cell's ID,

- updates internal paths,
- completes the orbital cascade (levels 1 to 10).

The Client's Private Torus becomes:

- living,
- oscillating,
- modified.

Transmission 4 — Client → Host

Confirmation of traversal execution

The Client returns the key after executing the traversal.

This informs the Host that:

- normal/ghost inversions were performed,
- the starting cell's ID mutated,
- internal paths were updated,
- fractal progression is coherent,
- the Client's Private Torus is up to date.

The Host can then:

- execute the same traversal,
- invert normal/ghost on its side,
- mutate the starting cell's ID,
- synchronize its Private Torus with the Client's.

Transmission 5 — Host → Client

Final synchronization confirmation

The Host sends the key one last time to confirm:

- it executed the traversal,
- it inverted normal/ghost,
- it mutated the starting cell's ID,

- its Private Torus is synchronized with the Client's,
- the session is directionally stable.

The Client receives this confirmation and the session becomes:

- coherent,
- stable,
- inviolable.

7.4. Why Five Transmissions?

7.4.1. Complete Bidirectional Validation

Host validates Client. Client validates Host. Then each validates the other's directional execution.

7.4.2. Protection Against Injections

An attacker cannot:

- intercept the key,
- modify it,
- resend it,
- falsify it.

Any incoherence is structurally detected.

7.4.3. Protection Against Repetitions

A repeated key:

- is detected,
- is rejected,
- destroys the session.

7.4.4. Protection Against Derivations

An attacker cannot derive:

- the starting point,
- the ending point,

- the 30 orbital bits,
- exponential movements,
- the targeted cell ID,
- ID mutation.

Directional structure is **non-derivable**.

7.4.5. Perfect Directional Synchronization

The 5 transmissions guarantee that:

- both Private Tori executed the same traversal,
- normal/ghost inversions are symmetrical,
- ID mutations are identical,
- internal geometry is coherent,
- no divergence is possible.

7.5. Initial Synchronization

Once the key is validated:

- the Authentication Torus is used once,
- the Private Torus is initialized,
- the global ID is created,
- normal/ghost pairs are defined,
- internal paths are activated,
- the torus unique ID is established,
- the Glass Fortress is armed.

This synchronization is **fractal**, not mathematical.

7.6. Initial Reconstruction of the Private Torus

The first key triggers:

- generation of the orbital ring,
- 60 orbital readings,

- 180 directional blocks (N,E),
- reading of 3-bit blocks in the Public Torus,
- mutation of the starting cell's ID,
- update of internal paths,
- establishment of directional coherence.

The Private Torus becomes:

- living,
- oscillating,
- operational.

7.7. Error Handling During the Handshake

7.7.1. Incoherent Key

If the key:

- contains an invalid ID,
- contains an impossible starting point,
- contains an impossible ending point,
- contains an illegitimate type,

→ the session is destroyed.

7.7.2. Modified Key

If the key returned by the Client differs:

- immediate destruction,
- Private Torus erased,
- IP invalidated.

7.7.3. No Response

If the Host receives no response after 5 seconds:

- the key is considered lost,
- a new key is generated,

- with a different starting point,
- but reconstructing the same directional file.

No retransmission of the same packet is ever performed.

7.8. Summary of Chapter 7

The directional handshake:

- relies on 5 transmissions,
- validates the first directional key,
- registers the Client's IP as trusted,
- initializes the Private Torus,
- activates the Glass Fortress,
- creates the global ID,
- establishes the torus unique ID,
- guarantees total directional coherence,
- protects against injections, repetitions, derivations, and falsifications.

It forms the foundation of **structural directional security**.

Chapter 8 — Multi-Torus and Parallel Networks

8.1. Overview

The multi-torus is one of the most powerful foundations of directional architecture. It allows the creation of multiple independent directional networks, each possessing:

- its own Private Torus,
- its own Authentication Torus,
- its own directional keys,
- its own internal IDs,
- its own global ID,
- its own unique torus ID,
- its own Glass Fortress.

The Public Torus is pre-generated. It comes with the protocol software and serves as the universal directional base for all branches.

Each parallel network is completely isolated from the others, even if they coexist:

- on the same machine,
- on the same system,
- in the same network environment.

The multi-torus enables:

- directional segmentation,
- structural sovereignty,
- total branch isolation,
- creation of inviolable private networks,
- controlled directional duplication.

8.2. Definition of a Parallel Network

A parallel network is a complete instance of directional architecture consisting of:

1. The Public Torus (pre-generated, universal)
2. An Authentication Torus
3. A Private Torus
4. A set of directional keys
5. A Glass Fortress
6. A global session ID
7. A unique torus ID

Each parallel network is an independent directional universe.

It shares:

- no keys,
- no IDs,
- no private torus,

- no fractal progression,
- no internal coherence.

8.3. Creation of a New Torus

Creating a new directional branch follows three steps:

8.3.1. Use of the Pre-Generated Public Torus

The Public Torus is not generated during branch creation. It is:

- pre-generated,
- provided with the software,
- universal,
- identical for all installations,
- non-modifiable,
- non-derivable.

It contains all possible combinations of the 30 bits, making it the universal directional base.

8.3.2. Generation of the Authentication Torus

An Authentication Torus of **1,000,000 cells × 3 bits** is generated. It validates the first directional key and initializes:

- the first normal/ghost pairs,
- the first internal paths,
- the global ID,
- the unique torus ID.

8.3.3. Generation of the Private Torus

A Private Torus of **1,000,000 cells × 3 bits** is generated. It is fractally reconstructed from directional keys:

- normal/ghost inversion,
- ID mutation,
- activation of internal paths,

- orbital progression.

Each Private Torus is independent from all other existing Private Tori.

8.4. Directional Branches

Each parallel network is called a **directional branch**.

A branch has:

- its own Private Torus,
- its own fractal progression,
- its own internal IDs,
- its own Glass Fortress,
- its own global ID,
- its own unique torus ID.

8.4.1. Independent Branches

Branches do not communicate with each other. They cannot:

- share keys,
- share IDs,
- share Private Tori,
- share internal paths,
- share ID mutations.

8.4.2. Sovereign Branches

Each branch is sovereign:

- it does not depend on any other branch,
- it cannot be influenced by another branch,
- it cannot be derived by another branch.

8.5. Directional Duplication (Offline Primary Torus)

A key feature of the multi-torus is **offline directional duplication**.

8.5.1. Primary Torus

A primary torus can be:

- generated,
- stored offline,
- transferred physically,
- used to create a new branch.

8.5.2. Offline Transfer

The primary torus can be transferred:

- on a USB drive,
- on a hard disk,
- on physical media,
- on any non-network medium.

This transfer contains **no directional keys**. It contains only:

- directional geometry,
- 3-bit blocks,
- internal paths.

8.5.3. Creating a Branch from the Primary Torus

Once the primary torus is transferred:

- a new branch is created,
- a new Private Torus is reconstructed,
- a new global ID is generated,
- a new unique torus ID is established,
- a new Glass Fortress is activated.

The branch is completely independent from the original branch.

8.6. Structural Isolation

Structural isolation is total.

8.6.1. Fractal Isolation

Fractal actions of one branch:

- cannot influence another branch,
- cannot derive another branch,
- cannot reconstruct another branch.

8.6.2. Directional Isolation

Internal paths of one branch:

- cannot be used in another branch,
- cannot be derived by another branch.

8.6.3. ID Isolation

Internal IDs:

- are not compatible between branches,
- cannot be transferred,
- cannot be derived.

8.7. Multi-Torus and Post-Quantum Security

The multi-torus strengthens post-quantum security:

- a quantum computer cannot derive a Private Torus,
- cannot derive a branch,
- cannot derive a key,
- cannot derive an ID,
- cannot derive the unique torus ID.

Even with infinite power, an attacker cannot:

- explore a Private Torus,
- derive a Private Torus,
- reconstruct a Private Torus,
- analyze a Private Torus.

Private Tori are **geometrically inviolable**.

8.8. Multi-Torus and Photonic Security

With the arrival of photonic computers:

- mathematical structures become obsolete,
- classical cryptography becomes vulnerable.

The multi-torus is designed for this era:

- Private Tori are non-transmissible,
- non-derivable,
- non-analyzable,
- non-explorable.

A photonic computer cannot:

- penetrate a Private Torus,
- analyze a Private Torus,
- derive a Private Torus.

8.9. Summary of Chapter 8

The multi-torus enables:

- creation of parallel networks,
- total branch isolation,
- offline directional duplication,
- structural sovereignty,
- post-quantum security,
- photonic security.

Each branch is an independent directional universe, inviolable by design. The pre-generated universal Public Torus serves as a common directional base, but **never** as a shared Private Torus.

Chapter 9 — Directional Ports and Internal Communication Geometry

9.1. Overview

TorusNet's directional ports are a radical extension of classical ports. Unlike TCP/UDP ports (0–65535), TorusNet uses:

- extended classical ports,
- several **billion** possibilities,
- mutable ports,
- non-scannable ports,
- non-derivable ports,
- non-falsifiable ports,
- ports structurally validated by the Glass Fortress.

A directional port is **not**:

- a simple integer,
- a socket,
- a TCP channel,
- a network identifier.

It is an **extended port**, generated by the Private Torus, which changes at each torus renewal and at each new connection.

9.2. Definition of a Directional Port

A directional port is:

- an extended port number (several billion possibilities),
- generated by the Private Torus,
- mutated at each torus renewal,
- mutated at each new connection,
- impossible to predict,
- impossible to scan,
- impossible to derive.

Structural rule: one connection per port

Each directional port:

- allows **only one** connection,
- is invalidated as soon as the session ends,
- is replaced by a new extended port at the next connection.

9.3. Why Directional Ports Are Inviolable

9.3.1. They do not exist in the classical network layer

An attacker cannot:

- scan ports,
- probe ports,
- open a port,
- send a packet to a port.

Directional ports exist **only** inside the torus's internal mechanics.

9.3.2. They are mutable and non-derivable

Directional ports:

- mutate automatically,
- are never reused,
- cannot be predicted,
- cannot be mathematically derived.

9.3.3. They are validated by the Glass Fortress

The Glass Fortress:

- monitors directional ports,
- validates their coherence,
- destroys the session if a port is violated,
- blocks the hostile IP in the operating system firewall.

9.3.4. They are non-transmissible

A directional port:

- cannot be sent,

- cannot be copied,
- cannot be exported,
- cannot be derived.

It exists **only locally**, inside the torus.

9.4. Mutation of Directional Ports

Directional ports mutate:

9.4.1. During torus renewal

Each time the Private Torus is regenerated:

- a new set of extended ports is generated,
- all previous ports become invalid.

9.4.2. During a new connection

At each new session:

- a new extended port is generated,
- only one connection is allowed on this port,
- any second connection attempt → **immediate destruction**.

9.5. Directional Ports and Transmission

Host → Client

The Host opens an extended directional port. The Client must respond on **that same port**.

Client → Host

The Client returns the key on this port. The Host validates coherence.

Host → Client

The Host sends the final key on the same port.

If a directional port is violated:

- the session is destroyed,
- the Private Torus is erased,
- the IP is blocked.

9.6. Directional Ports and Multi-Torus

In a multi-torus environment:

- each branch has its own directional ports,
- ports are not compatible between branches,
- ports cannot be derived from one branch to another.

A directional port is strictly **local** to its branch.

9.7. Directional Ports and Post-Quantum Security

Directional ports are inviolable by a quantum computer:

- impossible to derive an extended port,
- impossible to predict a mutation,
- impossible to explore the torus,
- impossible to reconstruct internal ports.

9.8. Directional Ports and Photonic Security

Photonic computers can break:

- RSA,
- ECC,
- TLS,
- SSH.

But they cannot:

- analyze directional geometry,
- derive a torus,
- reconstruct a directional port,
- predict a mutated extended port.

Directional ports are **structurally inviolable**.

9.9. Summary of Chapter 9

Directional ports:

- are classical ports extended to several billion possibilities,
- mutate at each torus renewal,
- mutate at each new connection,
- allow only one connection per port,
- are non-scannable,
- non-derivable,
- non-falsifiable,
- validated by the Glass Fortress,
- inviolable by design.

They form TorusNet's **internal communication layer**, completely independent from classical network protocols.

Chapter 10 — DNS and HTTPS Authentication (Temporary Trust Layer)

10.1. Overview

Before directional geometry takes full control of the session, a **temporary trust layer** is required to:

- identify the requested host,
- avoid malicious redirections,
- prevent classical MITM attacks,
- secure transmission of the first directional key,
- ensure the client is contacting the correct server.

This layer relies on two classical Internet technologies:

- **DNS** (domain name resolution),
- **HTTPS** (TLS certificate).

These technologies are **not** part of directional mechanics. They serve only as an entry gate before the protocol activates:

- the Authentication Torus,
- the Private Torus,

- the Glass Fortress,
- internal IDs,
- the global ID,
- the torus unique ID.

Once the first key is validated, DNS and HTTPS disappear completely.

10.2. Role of DNS in TorusNet

DNS is used for one reason:

Identify the requested host before entering directional geometry.

10.2.1. Domain verification

When a client contacts a server:

1. DNS resolves the domain name.
2. The protocol checks whether the domain belongs to the trusted host list.
3. If the domain is unknown or suspicious:
    - the connection enters user-confirmation mode,
    - no directional key is transmitted.

10.2.2. Protection against redirections

DNS prevents:

- malicious redirections,
- fake domains,
- cloned hosts,
- traffic hijacking.

10.2.3. Strictly temporary role

Once the first key is validated:

DNS is no longer used. Directional geometry takes over.

10.3. Role of HTTPS in TorusNet

HTTPS is used to:

Secure transmission of the first directional key.

10.3.1. Certificate verification

The client checks:

- the TLS certificate,
- domain validity,
- absence of classical MITM interception.

If the site lacks HTTPS:

- the connection enters user-confirmation mode,
- no directional key is sent.

10.3.2. Protection against classical attacks

HTTPS protects against:

- interceptions,
- injections,
- traffic manipulation,
- downgrade attacks.

10.3.3. Strictly temporary role

Once the first key is transmitted:

HTTPS disappears. The session becomes 100% directional.

10.4. DNS ↔ HTTPS ↔ Authentication Torus Interaction

The complete sequence is:

1. DNS identifies the host.
2. HTTPS secures the first transmission.
3. The Authentication Torus validates the first key.
4. The Glass Fortress activates the directional session.
5. The Private Torus takes control.
6. DNS and HTTPS are abandoned.

10.4.1. Full trust case

- DNS valid
- HTTPS valid → The first key is transmitted automatically.

10.4.2. Partial trust case

- DNS valid
- HTTPS absent → User confirmation required.

10.4.3. Suspicion case

- DNS suspicious
- HTTPS absent or invalid → Connection blocked until explicit validation.

10.5. Why TorusNet Still Uses DNS and HTTPS

TorusNet is a post-Internet directional architecture. However, today's ecosystem requires:

- domain names,
- TLS certificates,
- HTTP(S) servers.

DNS and HTTPS serve as a **bridge** between:

- classical Internet,
- directional Internet.

10.5.1. Final objective: complete disappearance

In future versions:

- DNS will be replaced by directional DNS,
- HTTPS will be replaced by native directional validation,
- certificates will be replaced by directional IDs,
- connections will be fully geometric.

10.6. Interaction with Directional Updates

Update 1 — ID inversion and mutation

Once the first key is validated:

- the starting cell's ID mutates,
- the global ID is created,
- DNS and HTTPS are no longer needed.

Update 2 — Autonomous directional session

The session becomes:

- continuous,
- living,
- autonomous.

Summary of Chapter 10

Chapter 10 establishes that:

- DNS and HTTPS only secure the first key,
- they disappear immediately after validation,
- the session becomes 100% directional,
- the Glass Fortress takes control,
- the Private Torus becomes the sole communication layer,
- internal IDs replace certificates,
- directional ports replace sockets,
- directional geometry replaces cryptography.

Chapter 11 — Long-Term Trust and Directional Reconnection

11.1. Overview

Directional reconnection must guarantee that the Private Torus truly belongs to the client and has not been:

- copied,
- stolen,
- transferred,
- cloned,

- moved to another machine.

To achieve this, TorusNet uses a **triple hardware identity**:

1. Motherboard ID
2. Processor ID
3. MAC address

This triple identity makes the Private Torus non-transferable and unusable elsewhere.

11.2. The Triple Hardware Identity

During reconnection, the client sends the Host:

1. Motherboard ID
2. Processor ID
3. Real MAC address

These three identifiers form a **directional hardware signature**, impossible to falsify.

11.3. Role of Each Identifier

11.3.1. Motherboard ID

- Engraved in firmware
- Non-modifiable
- Non-virtualizable

11.3.2. Processor ID

- Unique CPU identifier
- Non-clonable
- Non-simulable

11.3.3. MAC Address

- Hardware network identity
- Prevents torus usage on another interface

11.4. Why This Triple Identity Is Perfect

Even if an attacker copies:

- the Private Torus,
- directional files,
- extended ports,
- internal IDs,

they can never reproduce:

- the motherboard ID,
- the processor ID,
- the MAC address,
- the torus's fractal coherence.

11.5. Reconnection Sequence

During reconnection, the client sends:

- motherboard ID,
- processor ID,
- MAC address,
- reconnection network key,
- Private Torus ID,
- global session ID.

The Host compares the identifiers and either validates or destroys the session.

11.6. Directional Update of Hardware Identifiers

(New section — V5.3)

A user may change:

- their motherboard,
- their processor,
- their network card (thus their MAC).

To avoid losing their Private Torus, TorusNet allows **controlled directional updates** of hardware identifiers.

****11.6.1. Fundamental rule:**

To update one identifier, the user must prove identity using the other two.**

This yields three cases:

Case 1 — Updating the CPU ID

To update the CPU ID, the client must provide:

- motherboard ID,
- MAC address,
- old CPU ID,
- new CPU ID.

The Host checks:

- fractal coherence,
- directional coherence,
- hardware coherence.

If everything is coherent:

→ update authorized → Private Torus re-indexed → directional ports regenerated → CPU ID replaced

Case 2 — Updating the Motherboard ID

To update the motherboard ID, the client must provide:

- processor ID,
- MAC address,
- old motherboard ID,
- new motherboard ID.

Same logic:

→ directional validation → update authorized → Private Torus re-indexed

Case 3 — Updating the MAC Address

To update the MAC, the client must provide:

- motherboard ID,
- processor ID,
- old MAC,
- new MAC.

Same logic:

→ directional validation → update authorized → Private Torus re-indexed

11.6.2. Why This Rule Is Inviolable

An attacker cannot:

- simulate two coherent hardware identifiers,
- falsify two hardware identifiers simultaneously,
- reproduce the torus's fractal coherence,
- pass Glass Fortress validation.

Even with a stolen Private Torus, they can never provide:

- two valid hardware identifiers,
- coherent with each other,
- coherent with the Private Torus.

11.7. Interaction with the Glass Fortress

The Glass Fortress:

- monitors hardware identifiers,
- validates updates,
- detects falsifications,
- destroys the Private Torus if incoherent,
- blocks the hostile IP.

11.8. Summary of Chapter 11

Directional reconnection relies on:

- a triple hardware identity,

- fractal validation,
- directional coherence,
- impossibility of Private Torus transfer.

Hardware identifier updates are possible only if:

- two identifiers remain valid,
- fractal coherence is intact,
- the Glass Fortress validates the request.

This rule makes TorusNet:

- inviolable,
- non-transferable,
- resistant to theft,
- resistant to cloning,
- resistant to virtualization.

Update 6 — Version 6 of the TorusNet Protocol

Redesign of the Directional Seed and Introduction of the Multi-Layer Exponent Engine

This update defines the complete structure of the **70-bit directional seed**, used to generate internal trajectories within the private torus and the public torus. The exponent engine introduced in this version enables the representation of hyper-exponential values that cannot be inverted or derived by any known technology (classical, quantum, or photonic).

1. Structure of the Directional Key (128 bits)

The TorusNet directional key is composed of:

- **2 bits — Key type**
- **20 bits — Directional start point**
- **20 bits — Directional end point**
- **16 bits — Target cell ID**

- **70 bits — Directional seed**

Total:

$$128 \text{ bits}$$

2. Structure of the Directional Seed (70 bits)

The directional seed contains five components:

1. Exponential base (4 bits)
2. Fixed tower selection (4 bits)
3. Fixed tower height (2 bits)
4. Dynamic exponent layers (52 bits)
5. Final modulation values (addition + multiplication), encoded inside the seed (4 bits + 4 bits)

All components together form the complete 70-bit directional seed.

2.1 Exponential Base — 4 bits

The 4 bits define one of 16 hierarchical base values:

$$1,2,3,4,5,6,7,8,9,10,100,1000,10\,000,100\,000,1\,000\,000,10\,000\,000$$

2.2 Fixed Tower Selection — 4 bits

The 4 bits allow:

- activation of a fixed exponent tower (8 possible values),
- or selection of **no fixed tower** (8 possible values).

Available fixed towers:

$$1^{2^{3^4}},\ 2^{3^{4^5}},\ 3^{4^{5^6}},\ 4^{5^{6^7}},\ 5^{6^{7^8}},\ 6^{7^{8^9}},\ 7^{8^{9^{10}}},\ 8^{9^{10^{11}}}$$

2.3 Fixed Tower Height — 2 bits

Determines how many layers of the fixed tower are used:

- 00 → 1 layer
- 01 → 2 layers
- 10 → 3 layers

- 11 → 4 layers

2.4 Dynamic Layers — 52 bits

The **52 bits** are divided into **6 dynamic exponent layers**, each represented by a **raw 8-bit value**:

$$D_1, D_2, D_3, D_4, D_5, D_6 \in \{0, 1, \dots, 255\}$$

Directional rule:

- if a dynamic layer equals **0**, the layer is **ignored**,
- if its value is **> 0**, the layer is **active** in the exponent tower.

These 6 layers constitute the dynamic portion of the exponent engine.

2.5 Final Modulation (encoded inside the seed)

Two additional values are encoded inside the seed:

- **Addition (A)** — 4 bits → value from 0 to 15
- **Multiplication (M)** — 4 bits → value from 0 to 15

These 8 bits are part of the 70-bit seed and are applied **after** the full exponent structure is built.

The final exponent becomes:

$$E_{\text{final}} = (X + A) \times M$$

where **X** represents the full exponent tower:

$$X = \text{Base}^{\text{Fixed tower}^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}}$$

Rules:

- if **A = 0**, the addition has no effect,
- if **M = 0**, the final stage is **ignored**,
- if **M > 0**, the final stage is **active**.

3. Final Exponent Structure

With a fixed tower:

$$\text{Base}^{e_1^{e_2^{e_3^{e_4^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}}}}}$$

Without a fixed tower:

$$\text{Base}^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}$$

Then modulation:

$$E_{\text{final}} = (X + A) \times M$$

4. Representation Capacity

- **Raw space:**

$$2^{52}$$

- **Exponent space:** far beyond classical, quantum, and photonic computational limits.

5. Impact on the Private Torus

The private torus uses:

- the exponential base,
- the fixed tower (if activated),
- the 6 dynamic layers,
- the final modulation (A and M),
- the rule for ignoring zero-value layers,

to determine the internal directional trajectory.

Each seed produces a **unique direction**, impossible to predict or reproduce without the complete structure.

6. Conclusion of Update 6

Update 6 introduces:

- a **70-bit directional seed**,

- a hierarchical exponential base,
- an optional fixed tower,
- a configurable tower height,
- **6 dynamic layers of 8 bits**,
- a final modulation encoded inside the seed (**4-bit addition + 4-bit multiplication**),
- automatic suppression of zero-value layers.

This structure reinforces the **inviolable**, **non-derivable**, and **geometrically unpredictable** nature of the TorusNet protocol.